

Отчет по заданию «A2b. Анализ корректности FAST EXPONENT»

Тема: Инварианты цикла алгоритма быстрого возведения в степень

Выполнил: студент Ошаров Александр группы БПИ248

Семинарист: Маров Руслан

Аннотация

В отчете рассматривается алгоритм быстрого возведения в степень (FAST EXPONENT), дан анализ его вычислительной сложности по числу умножений, сформулирован инвариант цикла, а также проведена проверка его выполнения на этапах INIT, MNT и TRM. Показано, что данный метод обладает значительными преимуществами по сравнению с наивным способом.

1. Постановка задачи

Необходимо рассмотреть алгоритм быстрого возведения числа x в степень n и:

- 1) Определить точное количество умножений, выполняемых данным алгоритмом.
- 2) Сравнить алгоритм с наивным способом последовательного умножения.
- 3) Сформулировать условие P — инвариант цикла `while`.
- 4) Проверить выполнение инварианта на этапах INIT, MNT и TRM.

2. Алгоритм быстрого возведения в степень

Рассматривается следующий алгоритм:

```
int fastExponent(int x, int n) {
    int r = 1;
    int p = x;
    int e = n;

    while (e > 0) {
        if (e % 2 != 0) r = r * p;
        p = p * p;
        e = e / 2;
    }

    return r;
}
```

3. Анализ числа умножений

На каждой итерации цикла выполняются следующие операции:

- Одно умножение $r = r * p$.
- Одно дополнительное умножение $r = r * p$ в случае, если e нечетно.

Таким образом, количество умножений складывается из:

- 1) $\lfloor \log_2(n) \rfloor + 1$ операций возведения p в квадрат.
- 2) Числа единичных битов в двоичной записи n (так называемый $\text{popcount}(n)$) — операций обновления r .

Итого: всего требуется $\lfloor \log_2(n) \rfloor + 1 + \text{popcount}(n)$ умножений.

Наивный метод требует $(n-1)$ умножений, что существенно больше при больших n .

Алгоритм асимптотически эффективнее наивного метода, поскольку требует $O(\log n)$ умножений против $O(n)$. Однако при малых значениях n (например, $n \leq 3$) наивный способ может использовать меньше операций, так как быстрый алгоритм выполняет фиксированное количество действий на каждой итерации, включая возведение p в квадрат даже тогда, когда это избыточно. Таким образом, алгоритм не всегда лучше наивного, но значительно превосходит его при больших n .

4. Инвариант цикла

P : В любой момент выполнения цикла `while` верно равенство:

$$r * p^e = x^n.$$

Обоснование: изначально $r = 1$, $p = x$, $e = n$, следовательно $r * p^e = x^n$. Рассмотрим одну итерацию цикла. Пусть перед итерацией выполняется $r * p^e = x^n$.

- Если e чётно, то $e = 2k$. После присваиваний $p := p^2$, $e := k$ получаем $r * (p^2)^k = r * p^{2k} = r * p^e = x^n$.
- Если e нечётно, то $e = 2k + 1$. Тогда сначала $r := r * p$, затем $p := p^2$, $e := k$. Новое значение: $(r * p) * (p^2)^k = r * p^{2k+1} = r * p^e = x^n$.

Таким образом, инвариант сохраняется после каждой итерации.

5. Проверка выполнения инварианта

INIT: перед первой итерацией $r = 1$, $p = x$, $e = n$, следовательно $r * p^e = 1 * x^n = x^n$.

MNT: как показано в обосновании инварианта (раздел 4), после каждой итерации — как при чётном, так и при нечётном значении e — обновления переменных r , p и e сохраняют равенство $r * p^e = x^n$. Следовательно, инвариант поддерживается на каждой итерации цикла.

TRM: при завершении цикла $e = 0$, следовательно $r * p^0 = r = x^n$, что и возвращает алгоритм.

6. Заключение

Алгоритм быстрого возведения в степень требует $O(\log n)$ умножений, что значительно лучше наивного способа с $O(n)$ операциями. Инвариант $r * p^e = x^n$ выполняется на всех этапах выполнения цикла, что доказывает корректность алгоритма.